

Distributed Job Queue System — 50-Question Prep Guide

FastAPI • PostgreSQL • Redis Streams • SQLAlchemy 2.0 • Docker Compose • structlog

Part 1 — Concepts to know cold

Study these before drilling the questions. The interviewer flagged monitoring, security, and performance — those topics are marked.

Monitoring & observability INTERVIEWER FOCUS

Three pillars — metrics, logs, traces; monitoring vs. observability as a distinction.

Queue-specific metrics — queue depth, consumer lag, job age, throughput, failure rate, dead-letter queue size.

RED method (Rate, Errors, Duration) for the API; **USE** (Utilization, Saturation, Errors) for resources.

Prometheus + Grafana — /metrics endpoints, redis_exporter, postgres_exporter.

Liveness vs. readiness probes; worker heartbeats (alive vs. alive-but-stuck).

Correlation / trace IDs propagated API → queue → worker; OpenTelemetry.

SLI / SLO / error budget; alert on symptoms not causes; alert fatigue.

Security INTERVIEWER FOCUS

Threat modeling basics — assets, attackers, entry points.

AuthN vs. AuthZ — API keys, JWT, OAuth2/OIDC; multi-tenant isolation.

Secrets management — env vars vs. Vault / platform secrets; nothing in git or images.

Network posture — Redis/Postgres not publicly exposed, TLS, Redis AUTH, least-privilege DB users.

Container hardening — non-root user, minimal base image, image scanning (Trivy).

Supply chain — lockfiles, pip-audit / Dependabot, SBOM.

Input validation, payload size limits, rate limiting, what never to log (PII, secrets).

OWASP API Security Top 10.

Performance & scaling INTERVIEWER FOCUS

Latency percentiles (p50/p95/p99) vs. averages; throughput vs. latency trade-off.

Connection pooling — SQLAlchemy pool sizing, Redis pool, what exhaustion looks like.

Horizontal scaling via consumer groups; batching reads (XREADGROUP COUNT).

Backpressure and load shedding; unbounded stream growth (XTRIM / MAXLEN).

Load testing (k6, Locust) and capacity planning.

Async vs. sync workers — CPU-bound vs. I/O-bound jobs.

Architecture & distributed systems

Delivery semantics — at-least-once vs. at-most-once vs. "exactly-once"; idempotency.

Redis Streams internals — consumer groups, pending entries list (PEL), XAUTOCLAIM.

Retries — exponential backoff + jitter; dead-letter queues; poison messages.

Dual-write problem (Postgres + Redis) and the transactional outbox pattern.

Alternatives landscape — RabbitMQ, Kafka, SQS, Postgres-as-queue (FOR UPDATE SKIP LOCKED), Celery / Dramatiq / arq.

High availability — Redis Sentinel/Cluster, Postgres replication; single points of failure.

Graceful shutdown, crash recovery, job state machines.

Testing

Test pyramid — unit vs. integration vs. end-to-end; what belongs where.

Real dependencies vs. fakes vs. mocks — testcontainers; why fakeredis was removed.

Distributed failure modes — crash mid-job, duplicate delivery, race conditions.

Flaky tests in async suites; test isolation; what coverage doesn't tell you.

SDD & process

Design docs — functional + non-functional requirements, trade-off analysis, ADRs.

CI/CD — zero-downtime deploys, Alembic migration strategy, rollback, blue-green / canary.

Part 2 — The 50 questions

Check each box once you can answer it out loud without notes.

Architecture & design Q 1–12

- 01 ☐ Walk me through the full lifecycle of a job, from HTTP submission to result retrieval.
- 02 ☐ Why Redis Streams for the queue? Compare it to RabbitMQ, Kafka, SQS, and just using Postgres with SKIP LOCKED. When would each win?
- 03 ☐ Why do you need both Postgres and Redis? What problems does having two sources of state create?
- 04 ☐ You write to Postgres and Redis on submission — what happens if one succeeds and the other fails? How does the outbox pattern solve this?
- 05 ☐ What delivery guarantee does your system actually provide, and how did you verify that?
- 06 ☐ A worker is killed mid-job. What happens to that job, and how does it get picked up again?
- 07 ☐ What's a poison message, and how does your system stop one from being retried forever?
- 08 ☐ Describe your retry policy. Why exponential backoff, and why jitter?
- 09 ☐ What are the single points of failure here? How would you make each layer highly available?
- 10 ☐ When Redis and Postgres disagree about a job's state, which one wins and why?

11 ☐ How would you add job priorities, delayed jobs, or recurring/cron jobs to this design?

12 ☐ If you had to handle 100x the load, what's the first thing you'd redesign?

Performance & scaling

INTERVIEWER FOCUS Q 13-21

13 ☐ The system is slow — how do you find the bottleneck? What do you measure first?

14 ☐ Which do you scale first, API replicas or workers, and what tells you that?

15 ☐ Producers are enqueueing faster than workers consume. What happens today, and what should happen (backpressure, load shedding)?

16 ☐ How do you size the SQLAlchemy connection pool? What does pool exhaustion look like in production?

17 ☐ Why report p95/p99 latency instead of the average?

18 ☐ How would you load test this system, and what numbers would you put in the report?

19 ☐ Redis Streams grow unbounded — what's your trimming strategy and what breaks if you get it wrong?

20 ☐ How many jobs per second can your system handle right now? How would you find out?

21 ☐ When does async worker code hurt rather than help (e.g., CPU-bound jobs)?

Monitoring & observability

INTERVIEWER FOCUS Q 22-31

22 ☐ What's the difference between monitoring and observability?

23 ☐ What metrics matter most for a job queue system, in priority order?

24 ☐ How would you follow a single job across API → queue → worker in logs? (Correlation IDs, tracing.)

25 ☐ What alerts would you configure, and how do you avoid paging people at 3am for non-issues?

26 ☐ What SLOs would you define for this system? What's an error budget?

27 ☐ Liveness vs. readiness: what does each mean for the API vs. a worker? How do you detect a worker that's alive but stuck?

28 ☐ Design the on-call dashboard: what six panels go on it?

29 ☐ How would you detect jobs silently disappearing (accepted but never completed)?

30 ☐ How would you instrument this with Prometheus — what do you export from the API vs. the workers?

31 ☐ Where does structured logging (structlog) end and metrics begin — why not just grep logs?

Security

INTERVIEWER FOCUS Q 32-41

32 ☐ Threat-model this system: what are the assets, who are the attackers, what are the entry points?

33 ☐ The API currently has no auth — what would you add and why? API keys vs. JWT vs. OAuth2.

34 ☐ How would you isolate tenants so customer A can't query customer B's jobs?

35 ☐ Where do your DB/Redis credentials live today, and where should they live in production?

- 36 ☐ Redis and Postgres ports — who can reach them, and what's the right network posture?
-
- 37 ☐ A job payload contains attacker-controlled data. What could go wrong, and what limits/validation do you apply?
-
- 38 ☐ Harden your Docker images: what specifically would you change? (User, base image, scanning.)
-
- 39 ☐ How do you handle a CVE announced in one of your dependencies?
-
- 40 ☐ What must never appear in your logs, and how do you enforce that?
-
- 41 ☐ Which of the OWASP API Top 10 apply to this service?
-

Testing Q 42–47

- 42 ☐ What do you unit test vs. integration test in this system? Where's the boundary?
-
- 43 ☐ Mocks vs. fakes vs. real services in tests — why did you test against real Redis instead of fakeredis?
-
- 44 ☐ How would you write a test proving the system recovers from a worker crash?
-
- 45 ☐ How would you test for duplicate delivery and race conditions between concurrent workers?
-
- 46 ☐ What does 90% coverage not tell you about this system?
-
- 47 ☐ Your CI suite is flaky — what are the likely causes in an async/distributed test suite and how do you fix them?
-

SDD & process Q 48–50

- 48 ☐ Before writing code, what would a design doc for this system contain? Walk me through the non-functional requirements and one trade-off you'd document as an ADR.
-
- 49 ☐ How do you deploy this to production with zero downtime — schema migrations, worker draining, rollback?
-
- 50 ☐ How did you use AI while building this, and how did you verify its output was correct? (Expect this one — it's in your commit history.)
-

Suggested prep order

1. Monitoring & observability + Security (Q 22–41) — the interviewer's explicit focus.
2. Performance & scaling (Q 13–21) — the third flagged area.
3. Architecture & design (Q 1–12) — the "why did you pick X" backbone.
4. Testing, then SDD & process (Q 42–50).